

Lab Report (Analyze and verify the Growth Rate for Sorting Algorithms - B)

Name XXXXX

Course..... DSA

Lab Number..... 06

Lab Title..... Analyze and verify the Growth Rate for Sorting Algorithms - B

Date: 17-November-2025

Lab Level Attempted Basic

Objective:

To compare the performance of QuickSort Basic, QuickSort Better, QuickSort Practical, and Radix Sort on different input sizes. To analyze how pivot selection, hybrid sorting, and algorithm type affect execution time.

Description:

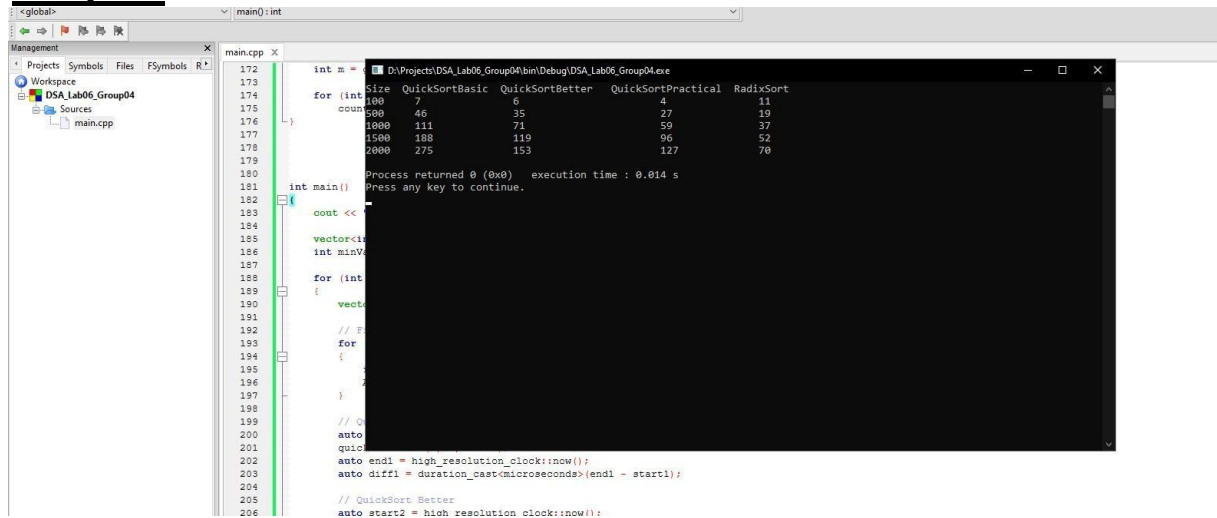
This lab compares the execution time of QuickSort variants and Radix Sort on different input sizes to study their efficiency and practical performance.

Conclusion:

QuickSort Better and Practical outperform QuickSort Basic due to improved pivot selection and hybrid optimization. Radix Sort is fastest for non-negative integers, showing the efficiency of non-comparison-based algorithms for specific data types.

Attachments:

Output:



The screenshot shows a C++ IDE with a project named 'DSA_Lab06_Group04'. The main.cpp file is open, showing a comparison of five sorting algorithms: QuickSortBasic, QuickSortBetter, QuickSortPractical, RadixSort, and another QuickSort. The code includes a table of counts for different input sizes (100, 500, 1000, 1500, 2000) and a main function that runs the algorithms and prints the execution time.

```
172 int m = 100;
173 for (int i = 0; i < m; i++)
174 {
175     count[100][0] = 7;
176     count[100][1] = 6;
177     count[100][2] = 4;
178     count[100][3] = 11;
179 }
180
181 int main()
182 {
183     cout << "Process returned 0 (0x0)   execution time : 0.014 s\n";
184     Press any key to continue.
185 }
186
187 // QuickSort Basic
188 // QuickSort Better
189 // QuickSort Practical
190 // Radix Sort
191 // QuickSort
192 // QuickSort Better
193 // QuickSort Practical
194 // Radix Sort
195 // QuickSort
196 // QuickSort Better
197 // QuickSort Practical
198 // Radix Sort
199 // QuickSort
200 // QuickSort Better
201 // QuickSort Practical
202 // Radix Sort
203 // QuickSort
204 // QuickSort Better
205 // QuickSort Practical
206 // Radix Sort
```

The output window shows the following table:

Size	QuickSortBasic	QuickSortBetter	QuickSortPractical	RadixSort
100	7	6	4	11
500	46	35	27	19
1000	111	71	59	37
1500	188	119	96	52
2000	275	153	127	70

Process returned 0 (0x0) execution time : 0.014 s
Press any key to continue.

Main.cpp

```
1 #include <iostream>
2 #include <string>
3 #include <vector>
4 #include <chrono>
5 using namespace std::chrono;
6 using namespace std;
7 // Utility function to swap two elements
8 void swap(int& a, int& b)
9 {
10     int temp = a;
11     a = b;
12     b = temp;
13 }
14
15 int partition(vector<int>& a, int left, int right)
16 {
17     // Choose random pivot
18     int pivotIndex = left + rand() % (right - left + 1);
19     int pivotValue = a[pivotIndex];
20
21     // Move pivot to end
22     swap(a[pivotIndex], a[right]);
23
24     // Move all smaller elements to the beginning
25     int storeIndex = left;
26     for (int i = left; i < right; i++)
27     {
28         if (a[i] < pivotValue)
29         {
30             swap(a[storeIndex], a[i]);
31             storeIndex = storeIndex + 1;
32         }
33     }
34
35     // Move pivot to its final place
36     swap(a[right], a[storeIndex]);
37     return storeIndex;
38 }
39
40 void quickSortBasic(vector<int>& a, int left, int right)
41 {
42     if (left < right)
43     {
44         int pivot = partition(a, left, right);
45         quickSortBasic(a, left, pivot - 1);
46         quickSortBasic(a, pivot + 1, right);
47     }
48 }
49 // Median-of-three function
50 void median3(vector<int>& a, int left, int right)
51 {
52     int center = (left + right) / 2;
53
54     if (a[center] < a[left])
55         swap(a[left], a[center]);
56     if (a[right] < a[left])
57         swap(a[left], a[right]);
58     if (a[right] < a[center])
59         swap(a[center], a[right]);
60
61     // Place pivot at position right - 1
62     swap(a[center], a[right - 1]);
63 }
64
65 int partitionBetter(vector<int>& a, int left, int right)
66 {
67     median3(a, left, right);
68     int pivot = a[right - 1];
69
70     int i = left;
71     int j = right - 1;
72
73     for (;;)
74     {
75         while (a[i] < pivot)
76             i++;
77         while (a[j] > pivot)
78             j--;
79         if (i < j)
80             swap(a[i], a[j]);
81     }
82     swap(a[i], a[right - 1]);
83     return i;
84 }
```

```

74     {
75         while (a[++i] < pivot) {}
76         while (pivot < a[--j]) {}
77
78         if (i < j)
79             swap(a[i], a[j]);
80         else
81             break;
82     }
83
84 // Restore pivot
85 swap(a[i], a[right - 1]);
86 return i;
87 }
88
89 void quickSortBetter(vector<int>& a, int left, int right)
90 {
91     if (left < right)
92     {
93         int pivot = partitionBetter(a, left, right);
94         quickSortBetter(a, left, pivot - 1);
95         quickSortBetter(a, pivot + 1, right);
96     }
97 }
98 void insertionSort(vector<int>& a, int left, int right)
99 {
100     for (int i = left + 1; i <= right; i++)
101     {
102         int temp = a[i];
103         int j = i - 1;
104
105         while (j >= left && a[j] > temp)
106         {
107             a[j + 1] = a[j];
108             j--;
109         }
110         a[j + 1] = temp;
111     }
112 }
113
114 void quickSortPractical(vector<int>& a, int left, int right)
115 {
116     if (left + 10 <= right)
117     {
118         // Use quicksort for large arrays
119         int pivot = partitionBetter(a, left, right);
120         quickSortPractical(a, left, pivot - 1);
121         quickSortPractical(a, pivot + 1, right);
122     }
123     else
124     {
125         // Use insertion sort for small arrays
126         insertionSort(a, left, right);
127     }
128 }
129
130 // A utility function to get maximum value in arr[]
131 int getMax(const vector<int>& arr)
132 {
133     int mx = arr[0];
134     for (int i = 1; i < arr.size(); i++)
135         if (arr[i] > mx)
136             mx = arr[i];
137     return mx;
138 }
139
140 // Counting sort according to the digit represented by exp
141 void countSort(vector<int>& arr, int exp)
142 {
143     int n = arr.size();
144
145     vector<int> output(n); // Output array
146     int count[10] = {0};
147
148     // Store count of occurrences
149     for (int i = 0; i < n; i++)
150         count[(arr[i] / exp) % 10]++;

```

```

151
152 // Change count[i] so that count[i] contains actual positions
153 for (int i = 1; i < 10; i++)
154     count[i] += count[i - 1];
155
156 // Build output array
157 for (int i = n - 1; i >= 0; i--)
158 {
159     int digit = (arr[i] / exp) % 10;
160     output[count[digit] - 1] = arr[i];
161     count[digit]--;
162 }
163
164 // Copy output back to arr[]
165 for (int i = 0; i < n; i++)
166     arr[i] = output[i];
167 }
168
169 // The main Radix Sort function
170 void radixsort(vector<int>& arr)
171 {
172     int m = getMax(arr);
173
174     for (int exp = 1; m / exp > 0; exp *= 10)
175         countSort(arr, exp);
176 }
177
178
179
180
181 int main()
182 {
183     cout << "Size QuickSortBasic QuickSortBetter\tQuickSortPractical RadixSort\n";
184
185     vector<int> sizes = {100, 500, 1000, 1500, 2000};
186     int minVal = 0, maxVal = 50;
187
188     for (int n : sizes)
189     {
190         vector<int> A(n), B(n), C(n), D(n);
191
192         // Fill all vectors with same random values
193         for (int i = 0; i < n; i++)
194         {
195             int val = minVal + rand() % (maxVal - minVal + 1);
196             A[i] = B[i] = C[i] = D[i] = val;
197         }
198
199         // QuickSort Basic
200         auto start1 = high_resolution_clock::now();
201         quickSortBasic(A, 0, n - 1);
202         auto end1 = high_resolution_clock::now();
203         auto diff1 = duration_cast<microseconds>(end1 - start1);
204
205         // QuickSort Better
206         auto start2 = high_resolution_clock::now();
207         quickSortBetter(B, 0, n - 1);
208         auto end2 = high_resolution_clock::now();
209         auto diff2 = duration_cast<microseconds>(end2 - start2);
210
211         // QuickSort Practical
212         auto start3 = high_resolution_clock::now();
213         quickSortPractical(C, 0, n - 1);
214         auto end3 = high_resolution_clock::now();
215         auto diff3 = duration_cast<microseconds>(end3 - start3);
216
217         // Radix Sort
218         auto start4 = high_resolution_clock::now();
219         radixsort(D);
220         auto end4 = high_resolution_clock::now();
221         auto diff4 = duration_cast<microseconds>(end4 - start4);
222
223         // Print results
224         cout << n << "\t"
225              << diff1.count() << "\t\t"
226              << diff2.count() << "\t\t\t"
227              << diff3.count() << "\t\t\t"

```

```
228         << diff4.count() << endl;
229     }
230
231     return 0;
232 }
```

Graphical Analysis of Quick Sort Variants and Radix Sort

Quick Sort with Random Pivot

Input Count (N)	Algorithm Running Time			Average Running Time
	Attempt 1	Attempt 2	Attempt 3	
100	10	8	7	8.333333333
500	48	47	59	51.33333333
1000	125	125	172	140.6666667
1500	187	185	164	178.6666667
2000	274	263	229	255.3333333

Quick Sort with Median of Three

Input Count (N)	Algorithm Running Time			Average Running Time
	Attempt 1	Attempt 2	Attempt 3	
100	8	6	6	6.666666667
500	36	37	82	51.66666667
1000	80	86	149	105
1500	115	116	104	111.6666667
2000	151	158	133	147.3333333

Quick Sort & Insertion Sort Hybrid

Input Count (N)	Algorithm Running Time			Average Running Time
	Attempt 1	Attempt 2	Attempt 3	
100	6	4	4	4.666666667
500	27	28	85	46.66666667
1000	66	77	83	75.33333333
1500	93	95	83	90.33333333
2000	126	127	115	122.6666667

Radix Sort

Input Count (N)	Algorithm Running Time			Average Running Time
	Attempt 1	Attempt 2	Attempt 3	
100	8	14	6	9.333333333
500	17	19	65	33.66666667
1000	41	36	61	46
1500	51	50	66	55.66666667
2000	69	69	64	67.33333333

Graphs

